Resume Deep-Dive: Q&A, Adaptive RAG, Slack Bolt, and a Deterministic Eval

Built from `Bidisha_Das_Resume_Crowdstrike.pdf`. Runnable code is in `resume_examples/`. The code is **illustrative**, not your production source. Say that if asked.

Chapter 1: Rules of engagement

Claims your own drill notes could not verify (from HM_ROUND_PREP chapter 6): the Bolt app repo, "150+ users across 8 teams", "20 min to under 3", "eight capabilities onto Headless 360", "~92%", "~35%", "~40% throughput". The ResolveFlow public repo showed OpenAI only, fewer scenarios than the resume's 50. Before the round, for each number either (a) be able to say how it was measured, or (b) switch to the mechanism and say "roughly" only if true.

Number defence line: "[Number] is [metric], measured from [start] to [end] in [system], over [window], [median/mean]. Caveat: [what else changed]."

The thread through every answer: *the model proposes; deterministic code authorises.* The resume says it twice (Salesforce bullet on error handling, ResolveFlow bullet on critique). Every Q&A below lands on it.

Chapter 2: Resume bullets and the questions they invite

Resume line	Likely question	Answer skeleton
Five agent workflows, ~60 days to ~5 days	"Walk me through one workflow end to end."	Name one (e.g. queue membership). Request in natural language → retrieve versioned Skill → ground in org metadata → plan → validate against allowlist → approval → execute as the user. Then the number line.
Retrieval-to-action architecture, versioned Skills	"Why Skills and not one big prompt?"	A Skill carries instructions plus a `tools:` list, so retrieval narrows both knowledge and the tool menu; versioning lets you roll back and evaluate one Skill alone.
Eight capabilities onto Headless 360, 10+ teams	"What does onboarding mean? Name them."	Mechanism: API exposure, tool discovery, MCP execution, with typed schemas and a permission check per op. Name them from memory or leave out the count.
MCP, REST, Metadata/Tooling API, schema validation, operation-level authz	"Where is authorisation enforced?"	In the tool layer on every call, with the caller's identity, never in the prompt or from a model-supplied argument.
Slack agent on Bolt, HITL blocks, requester's own permissions	"How does the identity work?"	See chapter 4. Slack user → mapped Salesforce user → call under that user's credentials; approval click verified as the requester.
Apex queue creation as a permission-checked tool	"Why Apex?"	Queue and membership operations sit on the platform; Apex gives a governed, testable entry point with inputs validated before DML.
Retired five legacy retrieval integrations	"What did you replace and what broke?"	Consolidated 20+ prompts, permission models, tool definitions. Be ready with one regression the replay suite caught.
Eval framework	"How do you know it works?"	Chapter 6. Layers, negative permission tests, replayed failures, per-category floors. Your measured figures: 524 utterances, 81.7% pass, 88.3% stable, kappa 0.649, no CI gate yet.
Error handling, graceful degradation, decisions deterministic in code	"What does the model NOT decide?"	Authorisation, allowlist membership, approve/escalate, which identity executes.
Cut misconfigured requests ~35%	"How measured?"	Needs a baseline window and a definition of "misconfigured". If you cannot give it, say so.
AWS, Docker, K8s, Splunk, ~40% throughput	"Walk me through deployment."	Use HM_ROUND_PREP 5.3. Do not equate latency gain with throughput.
ResolveFlow: interrupts + Postgres, OAuth, 50 scenarios, 86%	"How does pause/resume survive a restart?"	Checkpointer in Postgres keyed by thread id; `interrupt()` stores state; resume with `Command(resume=...)`. Writes refuse without a valid token. Settle the 50-scenario wording first.
AdaptiveRAG platform	"Why adaptive?"	Chapter 3.

Chapter 3: Adaptive and corrective RAG

3.1 The idea in 60 seconds

A fixed pipeline (embed, top-k, stuff into prompt) wastes tokens on easy questions and fails on hard ones. **Adaptive RAG** classifies the query first and picks a path: no retrieval (greeting, general knowledge), single-step retrieval (a factual lookup), or multi-step (decompose, retrieve per sub-question, merge). **Corrective RAG** grades what came back; if the evidence is weak it rewrites the query and retries a bounded number of times, and if it is still weak it abstains instead of guessing.

```
query → classify ─┬─ direct ──────────────────────────────→ answer
                  ├─ single ──→ retrieve → grade ─┬─ good ──→ answer (grounded)
                  └─ multi ───→ split → retrieve each → grade
                                                  └─ weak → rewrite (max N) → retrieve → still weak → abstain
```

Your resume numbers (+12% groundedness, -21% token cost, -18% median latency vs a fixed baseline, 100-question set): the cost and latency wins come from **skipping retrieval and reranking on easy queries**; the groundedness win comes from **grading and abstaining**. Say that, because it is the causal story an interviewer will probe.

3.2 Reference code (`resume_examples/adaptive_rag.py` , runs offline)

```
"""Adaptive + corrective RAG, minimal and runnable (no network, no LLM).

Illustrative. Stand-ins are marked: the classifier is rules (production: a small LLM or
fine-tuned classifier), scoring is token overlap (production: embeddings + FAISS + reranker).
"""
from __future__ import annotations

import re
from dataclasses import dataclass

CORPUS = {
    "doc1": "To reset a user password open Setup and choose Users then Reset Password.",
    "doc2": "Permission sets grant extra access on top of a profile and can be assigned to many
users.",
    "doc3": "Public groups can contain users roles and other groups and are used in sharing rules.",
    "doc4": "Queues own records such as cases and leads and have members who can accept them.",
}
RELEVANCE_MIN = 0.25      # below this the evidence may not influence the answer
MAX_REWRITES = 1          # corrective loop is bounded


def toks(s: str) -> set[str]:
    return set(re.findall(r"[a-z]+", s.lower()))


def classify(query: str) -> str:
    """Route: 'direct' (no retrieval), 'single' (one retrieval), 'multi' (decompose)."""
    q = query.lower()
    if re.fullmatch(r"(hi|hello|thanks|thank you)[!. ]*", q):
        return "direct"
    if " and " in q or " compare " in q or " versus " in q or q.count("?") > 1:
        return "multi"
    return "single"


def retrieve(query: str, k: int = 3) -> list[tuple[str, float]]:
    qt = toks(query)
    scored = [(d, len(qt & toks(t)) / (len(qt) or 1)) for d, t in CORPUS.items()]
    return sorted(scored, key=lambda x: x[1], reverse=True)[:k]


def grade(hits: list[tuple[str, float]]) -> list[tuple[str, float]]:
    """Corrective step: drop chunks under the threshold."""
    return [(d, s) for d, s in hits if s >= RELEVANCE_MIN]


def rewrite(query: str) -> str:
    """Stand-in for an LLM rewrite: strip filler words."""
    return " ".join(w for w in query.split() if w.lower() not in {"how", "do", "i", "the", "a", "can",
"please"})


@dataclass
class Result:
    route: str
    evidence: list[str]
    answer: str
    rewrites: int = 0


def answer(query: str) -> Result:
    route = classify(query)
    if route == "direct":
        return Result(route, [], "Hello! How can I help?")
    subqueries = [p.strip() for p in re.split(r"\band\b|\?", query) if p.strip()] if route == "multi"
else [query]
    evidence: list[str] = []
```

```
    rewrites = 0
    for sq in subqueries:
        good = grade(retrieve(sq))
        while not good and rewrites < MAX_REWRITES:        # corrective loop, bounded
            sq, rewrites = rewrite(sq), rewrites + 1
            good = grade(retrieve(sq))
        evidence += [d for d, _ in good]
    evidence = sorted(set(evidence))
    if not evidence:
        return Result(route, [], "I could not find supporting documents, so I won't guess.", rewrites)
    return Result(route, evidence, "Answer built from: " + ", ".join(evidence), rewrites)


def grounded(res: Result) -> bool:
    """Deterministic groundedness: every cited doc id exists in the corpus and is in the evidence."""
    cited = set(re.findall(r"doc\d+", res.answer))
    return cited <= set(res.evidence) and cited <= set(CORPUS)


if __name__ == "__main__":
    for q in ["hello", "reset password for a user", "permission sets and public groups", "banana
smoothie recipe"]:
        r = answer(q)
        print(f"{q!r:45} route={r.route:6} evidence={r.evidence} rewrites={r.rewrites} grounded=
{grounded(r)}")
```

Output you will see: greeting routes `direct`; "permission sets and public groups" routes `multi` and pulls doc2 and doc3; "banana smoothie recipe" tries one rewrite, finds nothing, and abstains.

3.3 Questions

Question	Answer
Naive vs reranked vs corrective vs adaptive?	Naive: top-k only. Reranked: retrieve wide, rerank with a cross-encoder, keep few; better precision, extra latency. Corrective: grade then rewrite or abstain. Adaptive: choose the path per query, so you pay only when needed.
How is the classifier built?	Small LLM or fine-tuned model with a few labels; evaluate it on its own (routing accuracy) because a wrong route silently degrades everything downstream.
What if the classifier is wrong?	Failure asymmetry: sending a hard query down `direct` is the costly error. Bias toward retrieving when unsure; log routes so you can audit.
How do you set the relevance threshold?	Labelled queries, sweep the threshold, plot precision and recall; pick by cost of a wrong answer. A fixed 0.7 or 0.25 means nothing without that curve.
How do you measure groundedness?	Offline: every claim must trace to a retrieved chunk (LLM-as-judge with a rubric, spot-checked by humans), plus deterministic citation checks. Say which one your +12% used.
Why bound the corrective loop?	Unbounded loops burn tokens and latency; one or two rewrites then abstain.
FAISS vs Pinecone?	FAISS: in-process library, no ops, you own persistence and scaling. Pinecone: managed, filtering and scale built in, a network hop and cost.
Chunking?	Size trades recall against context noise; overlap protects boundaries; keep metadata (source, version) for filtering and citations.
How is this different from the Salesforce retrieval?	There retrieval selected Skills and grounded plans in org metadata (hybrid search, rerank, permission filtering). AdaptiveRAG is a separate evaluation platform comparing strategies. Do not blur them.

Chapter 4: Slack Bolt, explained from zero

4.1 Mental model

Slack does not call your function; it sends **HTTP requests (Events API, interactivity) or WebSocket messages (Socket Mode)** to your app. Bolt is the framework that verifies the request signature, parses the payload, and routes it to a handler you register by event type, action id, slash command, or message pattern.

Concept	What it is
`app.event("app_mention")`	Handler for an Events API event (someone @-mentions the bot)
`app.action("approve")`	Handler for a Block Kit button click (interactivity)
`app.command("/x")`	Slash command
`ack()`	Tells Slack "received". **Must happen within 3 seconds** or the user sees an error and Slack retries
`say()` / `respond()`	Post a message / reply to an interaction (can replace the original message)
Block Kit	JSON UI: sections, buttons, selects. This is how approval blocks are built
Socket Mode vs HTTP	Socket Mode: app opens an outbound WebSocket, no public URL, good for dev and behind firewalls. HTTP: public endpoint, signature verification with the signing secret, what you scale behind a load balancer
Tokens	Bot token (`xoxb-`) for API calls; app-level token (`xapp-`) for Socket Mode; signing secret verifies inbound HTTP

4.2 The three-second rule, the part interviewers love

LLM calls take longer than 3 seconds. So: **ack immediately, do the slow work after, then post the result.** Events API: Bolt acks the HTTP request for you, but you should still hand slow work to a background worker or queue instead of blocking the handler. Interactions: call `ack()` first line. Slack **retries** unacknowledged requests, so handlers must be idempotent (dedupe on `X-Slack-Retry-Num` or event id, and on a request id in the button value).

4.3 Identity: "runs under the requesting user's own permissions"

1. Slack gives you a Slack user id (`U123`), which is not a Salesforce identity.
2. Map it to the Salesforce user (SSO email match or a stored link from an OAuth login). Unmapped means refuse.
3. Execute with **that user's** token or by impersonation the platform enforces, so Salesforce's own sharing and permission model is the real guard.
4. On the approval click, verify `body["user"]["id"]` equals the requester. Anyone in the channel can press a button; the click is untrusted input.
5. Re-check authorisation at execution time; permissions may have changed since the proposal.

4.4 Reference code (`resume_examples/slack_bolt_example.py`)

The pure helpers (`authorize` , `propose` , `execute`) run without Slack and are what the eval imports. `build_app()` wires them to Bolt.

```
"""Slack Bolt (Python) agent skeleton: ack fast, approve with Block Kit, run as the requester.

Illustrative. Run (needs a Slack app + tokens):
  SLACK_BOT_TOKEN=xoxb-... SLACK_APP_TOKEN=xapp-... uv run --with slack-bolt python
slack_bolt_example.py
The pure helpers below are testable without Slack; deterministic_eval.py imports them.
"""
from __future__ import annotations

import os
import uuid

ALLOWLIST = {"AddUserToQueue", "AssignPermissionSet"}                    # operation allowlist
SLACK_TO_SFDC = {"U_ALICE": "sf-alice", "U_BOB": "sf-bob"}               # real: SSO/identity lookup
SFDC_PERMS = {"sf-alice": {"AddUserToQueue"}, "sf-bob": set()}           # real: ask Salesforce, not a
dict
PENDING: dict[str, dict] = {}                                            # real: Redis/DB with TTL
DONE: set[str] = set()                                                   # idempotency


def authorize(sfdc_user: str | None, op: str) -> bool:
    """Deterministic gate. The model never decides this."""
    return sfdc_user is not None and op in ALLOWLIST and op in SFDC_PERMS.get(sfdc_user, set())


def approval_blocks(request_id: str, summary: str) -> list[dict]:
    return [
        {"type": "section", "text": {"type": "mrkdwn", "text": f"*Approval needed*\n{summary}"}},
        {"type": "actions", "block_id": "approval", "elements": [
            {"type": "button", "action_id": "approve", "style": "primary", "value": request_id,
             "text": {"type": "plain_text", "text": "Approve"}},
            {"type": "button", "action_id": "reject", "style": "danger", "value": request_id,
             "text": {"type": "plain_text", "text": "Reject"}},
        ]},
    ]


def propose(slack_user: str, op: str, args: dict) -> tuple[str, list[dict]] | tuple[None, str]:
    """Called after the LLM proposed (op, args). Code validates before the human is even asked."""
    sf = SLACK_TO_SFDC.get(slack_user)
    if not authorize(sf, op):
        return None, f"You are not allowed to run `{op}`."
    rid = str(uuid.uuid4())
    PENDING[rid] = {"slack_user": slack_user, "sf_user": sf, "op": op, "args": args}
    return rid, approval_blocks(rid, f"`{op}` with `{args}` as you ({sf})")


def execute(request_id: str, clicker: str) -> str:
    """Runs on Approve. Re-checks everything: the click is untrusted input."""
    if request_id in DONE:
        return "Already executed."                                       # double-click / Slack retry
    req = PENDING.get(request_id)
    if req is None:
        return "Request expired."
    if clicker != req["slack_user"]:
        return "Only the requester can approve."                         # no approving someone else's
request
    if not authorize(req["sf_user"], req["op"]):                         # permissions may have changed
        return "Not permitted any more."
    DONE.add(request_id)
    PENDING.pop(request_id, None)
    return f"Executed {req['op']} as {req['sf_user']}"                   # real: call API with THEIR
token


def build_app():
    from slack_bolt import App
    app = App(token=os.environ["SLACK_BOT_TOKEN"])
```

```
    @app.event("app_mention")
    def on_mention(event, say, ack=None):
        # Events API: Bolt acks the HTTP request for you; do slow LLM work off the request thread.
        op, args = "AddUserToQueue", {"queue": "Support", "user": "carol"}   # real: LLM proposal +
schema validation
        rid, blocks = propose(event["user"], op, args)
        if rid is None:
            say(blocks, thread_ts=event["ts"])
        else:
            say(blocks=blocks, text="Approval needed", thread_ts=event["ts"])

    @app.action("approve")
    def on_approve(ack, body, respond):
        ack()                                                              # MUST be within 3 seconds
        respond(replace_original=True, text=execute(body["actions"][0]["value"], body["user"]["id"]))

    @app.action("reject")
    def on_reject(ack, body, respond):
        ack()
        PENDING.pop(body["actions"][0]["value"], None)
        respond(replace_original=True, text="Rejected.")
    return app


if __name__ == "__main__":
    from slack_bolt.adapter.socket_mode import SocketModeHandler
    SocketModeHandler(build_app(), os.environ["SLACK_APP_TOKEN"]).start()
```

4.5 The flow to narrate

mention → Bolt receives event → handler maps user, asks the LLM for a proposal → **code** validates op and args and checks permission → posts Block Kit approval message with a request id → user clicks → `ack()` in under 3s → code re-verifies clicker, permission, and idempotency → executes as that user → `respond(replace_original=True)` with the result.

4.6 TypeScript equivalent of the handler shape (same ideas, you listed both languages)

```
import { App } from "@slack/bolt";
const app = new App({ token: process.env.SLACK_BOT_TOKEN, appToken: process.env.SLACK_APP_TOKEN,
socketMode: true });

app.event("app_mention", async ({ event, say }) => {
  const { id, blocks } = await propose(event.user!, "AddUserToQueue", { queue: "Support" }); //
validate in code
  await say({ blocks, text: "Approval needed", thread_ts: event.ts });
});

app.action("approve", async ({ ack, body, respond }) => {
  await ack();                                              // within 3 seconds
  const text = await execute((body as any).actions[0].value, body.user.id);
  await respond({ replace_original: true, text });
});
await app.start();
```

4.7 Questions

Question	Answer
What happens if the LLM call takes 10 s?	Ack first, process after, post the result by `say` / `chat.postMessage` or `chat.update`; never hold the interaction open.

Question	Answer
Why Socket Mode vs HTTP?	Socket Mode needs no public endpoint (dev, internal). Production at scale usually uses HTTP behind a load balancer with signature verification. State which you used.
How do you stop replay or forged requests?	HTTP: signing-secret verification plus timestamp window (Bolt does this). Buttons: request id in the value, single-use, bound to the requester.
How do you handle Slack retries?	Idempotency: dedupe on event id or retry header, and mark request ids as done before side effects.
Where does conversation state live?	Not in process memory. A durable store keyed by channel/thread; with LangGraph, the checkpointer keyed by thread id, so a click minutes later resumes the paused graph.
Who can approve?	Only the requester in the code above; for privileged ops a separate approver role check, enforced in code.
What are Slack rate limits?	Per-method tiers with `429` and `Retry-After`; back off and queue message updates.
Prompt injection via channel messages?	Message text is data. The model can only propose from an allowlisted menu, and code validates, so injected text cannot add a tool or change identity.
150+ users, 20 to under 3 minutes?	Know the instrument: where the 20 minutes was measured (request filed to access granted?), the sample, the date range. Mechanism-only fallback: removed the ticket and manual-implementation handoff.

Chapter 5: Is this your repo?

Your drill did **not** find a Bolt app in the repos it searched. The resume claims you shipped it. Before the round, find it (which repo, which language split between Python and TypeScript, which files), or reword the claim to what you can show. Prepare to be asked to open it and walk the `app_mention` and `approve` handlers.

Chapter 6: Deterministic evaluation

6.1 What "deterministic" buys you

An LLM-as-judge is noisy: same input, different score, and it drifts when the judge model changes. A **deterministic eval** checks things code can verify exactly, with no model in the loop, so the result is identical every run and can **gate CI**. Use it for everything that can be checked mechanically, and keep LLM judging only for subjective quality (helpfulness, tone), where you measure agreement (your kappa 0.649).

Layer	Deterministic check
Routing	`classify(query) == expected_route`
Retrieval	expected doc id appears in top-k (recall@k)
Tool selection	chosen tool is in the expected set
Permission enforcement	negative tests: a user without access must be refused; off-allowlist op refused; unmapped user refused
Structured output	JSON parses, exact keys, right types, op on allowlist
Groundedness (citation)	every cited id exists in the retrieved evidence
Abstention	no evidence → refuses instead of inventing
Replayed failures	recorded bad model outputs that once hit production, frozen as permanent regression cases

Run with a **recorded model output** (replay) or with the model stubbed, so the model is not in the test. That is what makes it deterministic.

6.2 Reference eval (`resume_examples/deterministic_eval.py`)

It imports the real functions from the two modules above, so it tests the actual code.

```
"""Deterministic eval: no LLM judge, no network, same result every run. Exit code gates CI.

Covers: routing accuracy, retrieval hit, permission enforcement (negative tests), structured
output validation, groundedness, and a replayed production failure.
Run: uv run python deterministic_eval.py
"""
from __future__ import annotations

import json
import sys

import adaptive_rag as rag
import slack_bolt_example as slack

ROUTING = [("hello", "direct"), ("reset password for a user", "single"),
           ("permission sets and public groups", "multi"), ("thanks", "direct")]
RETRIEVAL = [("how do I reset a user password", "doc1"), ("what do queues own", "doc4")]
PERMISSION = [  # (slack_user, op, expected_allowed) <- the negative cases matter most
    ("U_ALICE", "AddUserToQueue", True),
    ("U_BOB", "AddUserToQueue", False),        # user without access must be refused
    ("U_ALICE", "DeleteOrg", False),           # not on the allowlist
    ("U_UNKNOWN", "AddUserToQueue", False),    # unmapped Slack user
]
SCHEMA_REQUIRED = {"op": str, "args": dict}


def valid_proposal(raw: str) -> bool:
    """Structured-output contract: JSON, exact required keys with right types, op on allowlist."""
    try:
        obj = json.loads(raw)
    except json.JSONDecodeError:
        return False
    return (set(obj) == set(SCHEMA_REQUIRED)
            and all(isinstance(obj[k], t) for k, t in SCHEMA_REQUIRED.items())
            and obj["op"] in slack.ALLOWLIST)


# Replayed production failures: recorded model outputs that once broke something. Frozen forever.
REPLAYED = [
    ('{"op": "AddUserToQueue", "args": {"queue": "Support"}}', True),
    ('Sure! {"op": "AddUserToQueue"}', False),                  # prose around JSON
    ('{"op": "DropAllUsers", "args": {}}', False),              # hallucinated tool
    ('{"op": "AddUserToQueue", "args": "Support"}', False),     # wrong type
]


def run() -> dict[str, tuple[int, int]]:
    out = {}
    out["routing"] = (sum(rag.classify(q) == e for q, e in ROUTING), len(ROUTING))
    out["retrieval"] = (sum(rag.retrieve(q)[0][0] == d for q, d in RETRIEVAL), len(RETRIEVAL))
    out["permission"] = (sum(slack.authorize(slack.SLACK_TO_SFDC.get(u), op) == ok for u, op, ok in
PERMISSION), len(PERMISSION))
    out["schema_replay"] = (sum(valid_proposal(r) == ok for r, ok in REPLAYED), len(REPLAYED))
    r = rag.answer("banana smoothie recipe")                    # must abstain, not invent
    out["abstain_on_no_evidence"] = (int(r.evidence == [] and rag.grounded(r)), 1)
    return out


# Per-category floors. Permission is a hard 100%: one leak blocks the release.
FLOORS = {"routing": 0.9, "retrieval": 0.9, "permission": 1.0, "schema_replay": 1.0,
"abstain_on_no_evidence": 1.0}

if __name__ == "__main__":
    failed = False
    for cat, (ok, n) in run().items():
        rate = ok / n
        flag = "PASS" if rate >= FLOORS[cat] else "FAIL"
        failed |= flag == "FAIL"
```

```
        print(f"{flag}  {cat:24} {ok}/{n}  floor={FLOORS[cat]:.0%}")
    sys.exit(1 if failed else 0)
```

Actual output:

```
PASS  routing                 4/4  floor=90%
PASS  retrieval               2/2  floor=90%
PASS  permission              4/4  floor=100%
PASS  schema_replay           4/4  floor=100%
PASS  abstain_on_no_evidence  1/1  floor=100%
```

6.3 Design points to say out loud

- **Per-category floors, not one average.** A 95% average can hide a permission leak. Permission and schema are 100%; routing can tolerate 90%.
- **Negative tests matter most** for a security-first claim: prove the refusals, not just the successes.
- **Replay beats synthetic.** Each production failure becomes a frozen case, so the same bug cannot return.
- **Exit code gates promotion** (`sys.exit(1)` fails CI). Your own notes say no CI gate existed; present wiring this in GitHub Actions as the next step, not as done.
- **Honest limits:** deterministic checks cannot judge answer quality, and 4 to 6 cases per category here is a demo. Real sets are hundreds of cases, which is why the numbers (524, 81.7%) need a stated definition of "pass".

6.4 Questions

Question	Answer
Deterministic vs LLM-as-judge?	Deterministic for anything verifiable (schema, permission, routing, citations): exact, free, CI-gating. LLM judge for subjective quality, with a rubric and an agreement check against humans.
How do you test a non-deterministic agent deterministically?	Test the deterministic seams: classifier, validator, authoriser, retriever. Replay recorded model outputs through the parts after the model. Stub the model for the rest.
What does "stable 88.3%" mean?	Same input run repeatedly produced the same verdict 88.3% of the time; the remainder is model variance. Define it before you quote it.
How do you catch regressions after a prompt change?	Run the frozen suite; compare per-category rates against the last release; block if any floor is missed.
How do you benchmark models for cost and accuracy?	Same cases per model, record accuracy and tokens/cost per case; report accuracy per dollar. Your ResolveFlow claim (86% at 24% lower cost) needs the exact baseline and model list.
Where would this fail you?	Small case counts, cases written by the same person who wrote the code, and drift between the suite and real traffic. Fix with replayed production failures and sampling live traffic.

Chapter 7: Three-hour use of this guide

Time	Do
20 min	Chapter 1 and 5: decide the wording for each unverified number and find the Bolt repo
40 min	Chapter 4: read the code, then explain the flow and the 3-second rule aloud without looking
40 min	Chapter 3: explain adaptive vs corrective, then re-derive where each resume percentage came from
30 min	Chapter 6: run `uv run python deterministic_eval.py`, then add one new negative permission case and one replayed failure
30 min	Chapter 2 table as flashcards: answer each question aloud in 60 seconds

Run everything: `cd resume_examples && uv run python adaptive_rag.py && uv run python deterministic_eval.py`.